Ejercicio 1: Crear una lista enlazada simple desde cero

Prompt redactado:

Estructura solicitada: Implementación en Java de una lista enlazada simple de enteros desde cero (sin usar ArrayList ni LinkedList).

Explicación de clases y referencias:

- Crear una clase Nodo con atributos int dato y Nodo siguiente.
- Crear una clase ListaEnlazadaSimple con atributo Nodo head (referencia al primer nodo) y int size.
- Explicar que head inicia en null y que size incrementa en 1 con cada inserción.

Métodos a implementar:

1. insertarAlInicio(int dato): Crea el nodo y hace que su siguiente apunte al head actual, actualizando luego head al nuevo nodo.
2. insertarAlFinal(int dato): Si la lista está vacía, se asigna a head; si no, se recorre con un nodo auxiliar hasta el último nodo (actual.siguiente == null) y se enlaza.
3. imprimir(): Recorre imprimiendo los valores hasta llegar a null.
4. estaVacia(): Retorna head == null.
5. getSize(): Devuelve el atributo size.

Código y Pruebas: Método main demostrando inserciones al inicio y final, e imprimiendo la lista.

Ejercicio 2: Buscar elementos en una lista enlazada

Prompt redactado:

Estructura solicitada: Método boolean buscar(int dato) en la lista enlazada simple.

Justificación teórica: Explica en el prompt por qué no se puede acceder directamente por un índice como en un arreglo (acceso directo $O(1)$) y por qué exige obligatoriamente un recorrido secuencial ($O(n)$): los nodos están dispersos en memoria y la única forma de llegar al elemento k es siguiendo las referencias consecutivas desde head.

Lógica del método: Recorrer con un puntero Nodo actual = head mientras actual != null. Si actual.dato == dato, retornar true. Si termina el bucle sin encontrarlo, retornar false.

Código y Pruebas: Probar buscando un valor existente y uno inexistente en una lista poblada.

Ejercicio 3: Obtener un elemento por posición

Prompt redactado:

Estructura solicitada: Método int obtener(int posicion) en la lista enlazada.

Justificación y Validaciones:

- Explicar que para obtener la posición i, se debe iterar desde head realizando i saltos mediante actual = actual.siguiente.
- Validar posiciones inválidas: si posicion < 0 o posicion >= size, debe lanzar una excepción IndexOutOfBoundsException o mostrar mensaje de error.

Lógica del algoritmo: Avanzar un contador de 0 hasta posicion y retornar actual.dato.

Código y Pruebas: Probar obteniendo el elemento en la posición inicial (0), en una intermedia y validar el comportamiento ante un índice fuera de rango.

Ejercicio 4: Insertar un nodo en una posición específica

Prompt redactado:

Estructura solicitada: Método void insertarEnPosicion(int dato, int posicion).

Explicación crítica sobre el orden de enlaces:

- El orden correcto es:
    1. nuevo.siguiente = actual.siguiente;
    2. actual.siguiente = nuevo;
- Explicar detenidamente qué pasaría si se hace al revés (actual.siguiente = nuevo; primero): se pierde la referencia al resto de la lista enlazada, provocando la pérdida de nodos (memory leak / desconexión).

Casos a contemplar:

- posicion == 0: Llama a insertarAlInicio.
- posicion == size: Llama a insertarAlFinal.
- Posición intermedia: Busca el nodo previo (posicion - 1) y actualiza enlaces.
- Validar posiciones inválidas (posicion < 0 o posicion > size).

Código y Pruebas: Probar insertando en las tres posiciones (inicio, medio y final) e imprimir el resultado.

Ejercicio 5: Eliminar un nodo por valor

Prompt redactado:

Estructura solicitada: Método boolean eliminar(int dato) para eliminar la primera aparición de un valor.

Explicación del Garbage Collector en Java: Explica que en Java no existe la eliminación manual de memoria (como free() en C). Al "saltar" un nodo cambiando las referencias (anterior.siguiente = actual.siguiente), el nodo eliminado queda inalcanzable desde head y el Garbage Collector de Java se encarga de liberar la memoria automáticamente.

Casos límite a contemplar:

1. Lista vacía (head == null): Retorna false.
2. El dato está en el primer nodo (head.dato == dato): Hace head = head.siguiente.
3. Nodo del medio o final: Mantiene un puntero anterior para "saltear" a actual.
4. Dato inexistente: Recorre la lista completa sin encontrarlo y devuelve false.

Código y Pruebas: Probar la eliminación en cada uno de los 4 escenarios planteados.

Ejercicio 6: Eliminar un nodo por posición

Prompt redactado:

Estructura solicitada: Método void eliminarEnPosicion(int posicion).

Lógica del algoritmo y enlaces:

- Validar que posicion >= 0 y posicion < size.
- Si posicion == 0, hacer head = head.siguiente.
- Si es una posición intermedia o final, recorrer hasta el nodo en la posición posicion - 1 (nodo anterior).
- Reconectar: anterior.siguiente = anterior.siguiente.siguiente;.
- Decrementar size en 1.

Código y Pruebas: Eliminar el nodo inicial, un nodo intermedio y el último nodo, verificando la actualización del tamaño de la lista.

Ejercicio 7: Modificar un elemento de la lista

Prompt redactado:

Estructura solicitada: Método void modificar(int posicion, int nuevoDato).

Explicación teórica de la diferencia: Explica en los comentarios la diferencia conceptual entre modificar el valor interno almacenado en un nodo (actual.dato = nuevoDato) frente a modificar las referencias de la estructura (actual.siguiente = ...): modificar el dato preserva la topología y los enlaces intactos sin alterar la secuencia de la lista.

Lógica y Validación: Validar que la posición esté dentro del rango [0, size - 1], recorrer secuencialmente hasta el nodo de la posición y sobrescribir actual.dato.

Código y Pruebas: Modificar varios nodos y mostrar la lista antes y después de los cambios.

Ejercicio 8: Contar ocurrencias de un valor

Prompt redactado:

Estructura solicitada: Método int contarOcurrencias(int dato).

Justificación de recorrido completo: Explica en el prompt por qué el algoritmo no debe detenerse (break o return) al encontrar la primera coincidencia: para contabilizar la frecuencia exacta de un valor en un conjunto desordenado, se requiere obligatoriamente una inspección completa e ininterrumpida desde head hasta null.

Lógica: Inicializar un contador en 0, recorrer con actual != null, incrementar el contador en cada coincidencia y retornarlo al finalizar.

Código y Pruebas: Probar con la lista 10 -> 20 -> 10 -> 30 -> 10 -> null buscando el valor 10 (debe devolver 3).

Ejercicio 9: Invertir una lista enlazada simple

Prompt redactado:

Estructura solicitada: Método void invertir() para dar vuelta la secuencia de la lista in-place (ejemplo: 10 -> 20 -> 30 -> null se convierte en 30 -> 20 -> 10 -> null).

Explicación de punteros auxiliares y orden de actualización:

Explica el uso obligatorio de tres punteros auxiliares para no perder el rastro en memoria:

1. anterior = null
2. actual = head
3. siguiente = null

Bucle de inversión:

Java

```
while (actual != null) {
   siguiente = actual.siguiente; // Guardar el resto de la lista
   actual.siguiente = anterior;  // Invertir el puntero
   anterior = actual;            // Avanzar anterior
   actual = siguiente;           // Avanzar actual
}
head = anterior; // Reasignar la cabeza al nuevo primer nodo
```

Código y Pruebas: Imprimir la lista original, llamar a invertir() e imprimir el resultado invertido.

Ejercicio 10: Convertir la lista de enteros en una lista genérica

Prompt redactado:

Estructura solicitada: Convertir la implementación en una lista genérica ListaEnlazada<T> usando un Nodo<T>.

Justificación teórica de Generics: Explica qué partes cambian y cuáles permanecen iguales:

- Cambia: La firma de la clase (<T>), el tipo del atributo dato (T dato) y los parámetros/retornos de los métodos.
- Permanece igual: Toda la lógica de punteros (head, siguiente), recorridos secuenciales, inserciones y eliminaciones, demostrando que la manipulación estructural es independiente del tipo de dato contenido.
- En comparaciones de igualdad usar .equals() en lugar de ==.

Código y Pruebas: Probar la misma clase con ListaEnlazada<Integer>, ListaEnlazada<String> y una lista con objetos personalizados (ejemplo: ListaEnlazada<Alumno>).